Compresión de archivos en el editor de texto

Documentación técnica del módulo de compresión Huffman concurrente integrado al editor `eafitOS`. Para la visión general del proyecto y la justificación de las decisiones del editor, ver `README.md`.

Visión general

El compresor toma el documento que el editor tiene en memoria y lo guarda en disco en un archivo `.huf` más pequeño, usando **codificación de Huffman**. El trabajo de contar y codificar se reparte entre varios hilos. Mientras tanto, **el editor espera**: el comando `comp` no devuelve el prompt hasta terminar y muestra una barra de progreso. Como el editor queda detenido, el usuario puede **cancelar con Ctrl+C** en cualquier momento sin perder el documento ni dejar archivos a medias. La sección La integración con el editor explica por qué se eligió así y no en segundo plano, que fue la primera versión, y cómo funciona la cancelación.

```
editor>  comp notas.huf
           │
           ▼
   buf_serializar ──► bytes contiguos del documento
           │
           ▼
   comprimir.c ─── fase 1: N hilos cuentan frecuencias
           │       fase 2: se construye el árbol (una vez)
           │       fase 3: N hilos codifican bloques de 64 KB
           │       fase 4: el hilo que llamó escribe los bloques EN ORDEN ──► barra
           │                                          (y revisa si hubo Ctrl+C)
           ▼
   notas.huf.tmp ──rename──► notas.huf  (formato HUF2)
```

La idea de Huffman, en simple

Normalmente cada carácter ocupa 8 bits, aparezca mucho o poco. Huffman dice: *a los caracteres frecuentes, códigos cortos; a los raros, códigos largos*. Es la misma idea del código Morse, donde la letra más común es un solo punto.

Con el texto `aaaabbc`:

Carácter	Frecuencia	Código	Bits usados
`a`	4	`0`	4 × 1 = 4
`b`	2	`10`	2 × 2 = 4
`c`	1	`11`	1 × 2 = 2

Total: **10 bits** en lugar de 7 × 8 = **56 bits**.

Como todos los símbolos quedan en hojas del árbol, **ningún código es prefijo de otro**. Por eso la secuencia se decodifica sin separadores: se baja por el árbol y al tocar una hoja termina un símbolo.

Las capas del sistema

El compresor está dividido en cuatro capas, cada una ignorante de la de arriba. Encima de ellas, el editor es solo un cliente:

Capa	Archivos	Responsabilidad	¿Sabe de hilos?
cliente	`cat_editor.c`	Comandos `comp` y `desc`, barra de progreso, manejador de Ctrl+C	No crea hilos: solo llama y espera
3	`comprimir.c/h`	Repartir el trabajo entre hilos, escritor ordenado	**Sí** (único archivo con `pthread`)
2	`huf_formato.c/h`	Formato del archivo `.huf`, llamadas al sistema	No
1	`huffman.c/h`	Árbol y tabla de códigos	No
0	`bitio.c/h`	Empaquetar y desempaquetar bits	No

Esta separación no es estética. Las capas 0 y 1 **no cambiaron** al paralelizar el programa: se escribieron sin estado global, recibiendo y devolviendo todo por parámetro, y eso bastó para que varios hilos pudieran usarlas a la vez. La capa 2 sí recibió ajustes, pero ninguno la hizo depender de `pthread`:

Cambio en `huf_formato.c`	Para qué
Separar `huf_comprimir` en `huf_comprimir` + `huf_comprimir_con_freq`	Que el compresor paralelo reutilice el mismo ensamblado del archivo con las frecuencias contadas por varios hilos
Añadir el callback de progreso (`HufProgreso`)	Informar el avance sin que el módulo sepa quién dibuja la barra ni si hay hilos
Pasar del formato HUF1 a HUF2	Que los archivos pequeños no crezcan (ver la disposición del archivo)
Hacer que el callback devuelva `int` y añadir `HUF_CANCELADO`	Que el editor pueda pedir la cancelación sin que el módulo conozca las señales

La concurrencia se montó encima, en `comprimir.c`, en lugar de inyectarse en cada módulo.

Después llegó la auditoría (ver `AUDITORIA.md`), que sí modificó las tres capas bajas, pero por rendimiento y seguridad, no por concurrencia. Ninguno de esos cambios alteró el formato: el `.huf` resultante es **idéntico byte a byte** al de antes.

Capa	Cambio de la auditoría	Efecto
0 `bitio.c`	`bw_bits` escribe hasta 8 bits por iteración en vez de uno	Menos iteraciones en el bucle más caliente de la compresión
1 `huffman.c`	`mont_extraer` no puede indexar fuera del arreglo con el montículo vacío	Elimina un aviso legítimo del compilador; mismo orden de elementos
2 `huf_formato.c`	Decodificación con una tabla de búsqueda de 12 bits	Descomprimir 64 MB pasó de 2,34 s a 0,53 s junto con `-O2`
todas	Compilar con `-O2 -g` (antes `-O0`)	Comprimir 64 MB con 1 hilo pasó de 3,35 s a 1,84 s

Lo que cambió respecto a la primera versión. Había una quinta capa, `tarea_comp.c`, que ejecutaba la compresión en un hilo de fondo y llevaba su estado con un mutex. Se eliminó cuando se decidió que el editor espere mientras comprime (ver La integración con el editor). Que se pudiera quitar una capa entera sin tocar las de abajo es la mejor prueba de que la separación funcionaba.

Lo que hacen bitio.c, bitio.h y test_bitio.c

El problema que resuelven

Huffman produce códigos como `0`, `110`, `100`: **secuencias de bits de longitud variable**. Pero los archivos no se escriben bit por bit, la unidad mínima que acepta `write()` es un byte.

Si codificas `a` (1 bit) y luego `c` (3 bits), llevas 4 bits, que no son ni medio byte. Hace falta algo que vaya **acumulando bits sueltos hasta completar bytes**, y su contraparte que haga lo inverso al leer. Eso es todo lo que hace este módulo.

Es la pieza más aburrida del proyecto y la que más errores esconde. Por eso se escribió primero y con pruebas propias.

Cómo funciona por dentro

El escritor tiene un byte "en construcción" llamado `acum` y un contador `nbits_acum` de cuántos bits lleva metidos. Cada bit nuevo entra en la posición libre más a la izquierda. Cuando se llenan los 8, el byte se guarda en el buffer y `acum` vuelve a cero.

```
bits escritos:    1  0  1  1  0  0  1  0
posición:         7  6  5  4  3  2  1  0
byte resultante:  0xB2
```

Si solo se escriben 3 bits, el byte se cierra rellenando con ceros a la derecha (`0xE0`), pero el contador `total_bits` sigue valiendo 3. Ese número es lo que después permite al lector ignorar el relleno.

bitio.h — el contrato

Declara dos estructuras. El escritor:

```
typedef struct {
    unsigned char *datos;      /* buffer de salida */
    size_t         cap;        /* bytes reservados */
    size_t         nbytes;     /* bytes ya cerrados */
    unsigned char  acum;       /* el byte en construcción */
    int            nbits_acum; /* cuántos bits lleva acum (0..7) */
    uint64_t       total_bits; /* bits escritos en total */
} BitWriter;
```

Los campos que importan son los dos últimos. `acum` y `nbits_acum` son el byte a medio llenar: son el estado que hace que este módulo no sea trivial. Y `total_bits` es el que se va a guardar en la cabecera del `.huf`, el único dato que le permite al descompresor saber dónde termina el contenido real y dónde empieza el relleno.

La decisión de diseño que define el módulo

El escritor acumula en un **buffer de memoria**, no en un descriptor de archivo.

Es lo que hace posible toda la concurrencia posterior. Cada hilo trabajador tiene su propio `BitWriter` privado y codifica su bloque sin compartir absolutamente nada con los demás: cero mutex, cero contención. Un único hilo escritor toma después esos buffers ya terminados y los vuelca al archivo en el orden correcto.

Si este módulo recibiera un `fd`, todos los hilos competirían por la posición del archivo y habría que serializarlos: el paralelismo desaparecería.

El lector

El `BitReader` es la pieza que **deshace** lo que hace el escritor: toma los bytes empaquetados y vuelve a sacar los bits uno por uno, en el mismo orden en que se escribieron.

¿Para qué sirve el lector?

Sin él no hay descompresión. El archivo `.huf` es una ristra de bytes sin sentido aparente; para recuperar el texto original hay que leer bit por bit y bajar por el árbol de Huffman: `0` a la izquierda, `1` a la derecha, y cuando se llega a una hoja, ese es un carácter. Se vuelve a la raíz y se sigue.

El lector es lo que alimenta ese recorrido.

Y tiene un segundo uso, en la práctica más importante: **es lo que permite probar el escritor**. La única forma de saber que se empaquetaron bien 100000 bits es desempaquetarlos y comparar. Por eso los dos se escriben juntos.

Qué guarda

Tres campos, y ninguno sobra:

```
const unsigned char *datos;   /* los bytes a leer (no son suyos) */
uint64_t total_bits;          /* cuántos bits son de verdad */
uint64_t pos_bit;             /* por dónde va */
```

`pos_bit` es un contador **global de bits**, no de bytes. Empieza en 0 y avanza de uno en uno. Del bit 0 al 7 está en el primer byte, del 8 al 15 en el segundo, y así.

`total_bits` es el límite, el número que el escritor dejó y que viaja en la cabecera del archivo. Es lo que evita leer el relleno.

El `const` del primer campo no es adorno: declara que el lector **no modifica** los datos, lo que hace seguro que varios hilos lean el mismo buffer a la vez. El lector tampoco es dueño de esa memoria: no hace `malloc` ni tiene función de liberar.

Cómo encuentra un bit

Toda la lógica está en tres líneas:

```
        uint64_t indice_byte = r->pos_bit / 8;
        int      desp        = r->pos_bit % 8;
        return (r->datos[indice_byte] >> (7 - desp)) & 1;
```

La división entera dice **en qué byte** está el bit, y el residuo **en qué posición** dentro de ese byte. El `7 - desp` respeta la convención: como el primer bit se guardó en la posición 7, el bit número `desp` del byte está en la posición `7 - desp`.

Desplazar a la derecha lleva ese bit hasta la posición 0, y el `& 1` borra todo lo demás.

Cuando `pos_bit` llega a 8, la división cambia de byte automáticamente. No hay ningún caso especial que programar: la aritmética se encarga.

La guarda que hace todo esto seguro

```
        if (r->pos_bit >= r->total_bits) return -1;
```

Esa línea es la que hace que el relleno sea invisible. Si el escritor puso 3 bits y rellenó 5 ceros, el lector devuelve los 3 y al cuarto intento responde `-1`. Nunca llega a mirar los ceros sobrantes.

También protege contra leer fuera del buffer. Sin ella, pedir bits de más leería memoria ajena: comportamiento indefinido y una vulnerabilidad de manual.

bitio.c — las cuatro líneas que hay que entender

El resto es rutina. Estas cuatro son el corazón.

Meter un bit en su posición.

```
        w->acum |= (unsigned char)(0x80u >> w->nbits_acum);
```

`0x80` es `10000000` en binario, o sea el bit 7 encendido. Desplazarlo a la derecha `nbits_acum` veces lo mueve a la posición que toca: con 0 bits acumulados queda en el bit 7, con 1 en el bit 6, con 2 en el bit 5. El `|=` lo enciende sin tocar los que ya estaban.

Solo se ejecuta si el bit es 1. Cuando es 0 no hay nada que hacer, porque `acum` arrancó en cero.

Escribir n bits del más significativo al menos.

La primera versión escribía de a un bit:

```
        for (int i = n - 1; i >= 0; i--)
            bw_bit(w, (valor >> i) & 1);
```

El bucle va **hacia atrás** a propósito. Para escribir el código `110`, primero hay que sacar el bit de la posición 2, luego el 1, luego el 0. Si el bucle fuera hacia adelante saldría `011`. Ese es el error clásico, y por eso hay una prueba que fija la convención.

El problema era la velocidad: `bw_bits` se ejecuta **una vez por cada byte del archivo**, así que es el bucle más caliente de la compresión, y con códigos de 3 a 12 bits hacía de 3 a 12 vueltas por símbolo. Desde la auditoría mete de una vez todos los bits que caben en el byte en construcción:

```
        while (n > 0) {
            int libres = 8 - w->nbits_acum;                /* huecos en el byte actual    */
            int k      = n < libres ? n : libres;          /* cuántos bits caben ahora    */
            unsigned trozo = (unsigned)(valor >> (n - k)) & ((1u << k) - 1u);  /* los k de más arriba */

            w->acum       |= (unsigned char)(trozo << (libres - k));  /* a su posición      */
            w->nbits_acum += k;
            w->total_bits += (uint64_t)k;
            n             -= k;

            if (w->nbits_acum == 8) { /* cerrar el byte */ }
        }
```

Ejemplo: escribir `110` (3 bits) cuando el byte en construcción ya tiene 6 bits.

```
vuelta 1: libres = 2, k = 2 → trozo = "11"  → el byte se completa y se cierra
vuelta 2: libres = 8, k = 1 → trozo = "0"   → entra en la posición 7 del byte nuevo
```

Dos vueltas en lugar de tres, y con códigos cortos casi siempre una. La convención no cambió (sigue saliendo primero el bit más significativo), así que `test_bitio` sigue pasando y el `.huf` es idéntico.

El relleno.

```
        if (w->nbits_acum == 0) return 0;   /* nada pendiente */
        w->datos[w->nbytes++] = w->acum;    /* los bits que faltan quedan en cero */
```

`total_bits` **no se incrementa aquí**. Es la línea más importante del archivo: el relleno se escribe al disco pero no se cuenta como contenido.

El límite del lector.

```
        if (r->pos_bit >= r->total_bits) return -1;
```

Escritor y lector se coordinan **solo** a través de ese número.

El crecimiento del buffer

```
        size_t nueva = w->cap ? w->cap : CAP_POR_DEFECTO;
        while (nueva < necesarios) nueva *= 2;

        unsigned char *tmp = realloc(w->datos, nueva);
        if (tmp == NULL) { perror("realloc"); return -1; }
        w->datos = tmp;
```

La capacidad se **duplica**, no crece de a uno. Creciendo de a un byte, escribir n bytes costaría n reallocs con posible copia completa: $O(n^2)$. Duplicando, son $\log_2(n)$ reallocs y el costo amortizado por byte es constante.

La variable `tmp` no es cosmética: si `realloc` falla devuelve `NULL` **sin liberar el bloque anterior**, y asignarlo directo perdería el buffer entero.

test_bitio.c — qué demuestra cada prueba

No son pruebas decorativas; cada una atrapa un error concreto.

Prueba	Error que atrapa
Los 8 bits dan `0xB2`	La convención MSB primero está invertida
Relleno: `0xE0` con `total_bits = 3`	El relleno se cuenta como contenido, o el lector lo devuelve
Anchos 0, 3, 64, 1	`bw_bits` falla en los extremos o desborda con 64 bits
Escritor vacío	Un archivo vacío hace fallar el programa
100000 bits aleatorios	Cualquier error de empaquetado; valida ~11 reallocs
3 bloques independientes	Los bloques no son autónomos

La última es la que conecta con lo que viene. Crea tres `BitWriter` separados, cada uno con su contenido y su propio `flush`, y los decodifica por separado. **Eso es exactamente lo que van a hacer los hilos**: cada uno codifica su bloque en su propio escritor, sin compartir estado. Si esa prueba pasa, la arquitectura concurrente es viable.

La de los 100000 bits usa `srand(12345)` con semilla fija a propósito: si falla, falla siempre igual y se puede depurar. Una prueba aleatoria no reproducible es inútil.

Resultado: 18 pruebas, 18 superadas.

Lo que hacen huffman.c, huffman.h y test_huffman.c

Qué hace el archivo, en simple

Una sola cosa: **decidir qué secuencia de bits le toca a cada byte**. Recibe un conteo de frecuencias y devuelve una tabla que dice "la letra `a` es `0`, la `c` es `100`". No lee archivos, no escribe archivos, no sabe que existen los hilos. Es matemática pura sobre un arreglo de 256 números.

Entrada:

```
freq['a'] = 5     freq['b'] = 2     freq['c'] = 1
freq['d'] = 1     freq['r'] = 2
```

Salida:

```
tabla['a'] = código 0,   longitud 1
tabla['b'] = código 110, longitud 3
tabla['c'] = código 100, longitud 3
tabla['d'] = código 101, longitud 3
tabla['r'] = código 111, longitud 3
```

Eso es a propósito: al no tener estado global ni efectos secundarios, se puede probar de forma aislada y varios hilos pueden usarlo a la vez sin ningún mecanismo de sincronización.

Lo que NO hace: no comprime nada. Solo arma el diccionario. Quien lo usa para recorrer el archivo y escribir bits es `huf_formato.c`.

huffman.h — el contrato

```c
typedef struct {
    uint64_t codigo;    /* los bits, alineados a la derecha */
    uint8_t  longitud;  /* cuántos son significativos; 0 = símbolo ausente */
} Codigo;

typedef struct NodoHuf {
    uint64_t freq;
    uint64_t orden;     /* desempate determinista */
    int      simbolo;   /* 0..255 en hojas; -1 en internos */
    struct NodoHuf *izq, *der;
} NodoHuf;

typedef struct {
    NodoHuf *nodos;       /* arena: TODOS los nodos en un solo bloque */
    size_t   n_nodos;
    NodoHuf *raiz;        /* NULL si el archivo está vacío */
    Codigo   tabla[256];
    int      n_distintos;
} Huffman;
```

La tabla necesita **los dos campos**. Sin `longitud`, los códigos `101` y `0101` se guardarían como el mismo número 5 y serían indistinguibles.

1. El montículo

El algoritmo necesita sacar repetidamente los dos nodos de menor frecuencia. Se podría recorrer la lista entera cada

vez buscando el mínimo —O(n) por búsqueda— pero un montículo binario lo hace en O(log n).

Un montículo es un árbol binario donde cada padre es menor o igual que sus hijos, con un truco: **no se guarda como árbol, se guarda como arreglo**. La relación padre-hijo sale de la aritmética de los índices.

Por eso en el código aparecen esas fórmulas:

```
size_t padre = (i - 1) / 2;              /* en subir()  */
size_t izq = 2*i + 1, der = 2*i + 2;     /* en bajar()  */
```

No hay punteros ni nodos de árbol: el "árbol" existe solo en la aritmética. Eso lo hace rapidísimo porque todo está contiguo en memoria.

`subir` se usa al insertar. Se mete el nodo nuevo al final y se va intercambiando con su padre mientras sea menor. Sube hasta encontrar su lugar.

`bajar` se usa al extraer. Se saca la raíz (que es el mínimo), se pone el último elemento en su lugar, y se hunde intercambiándolo con el menor de sus hijos hasta que la propiedad se restaure.

Ambos recorren a lo sumo la altura del árbol, que es $\log_2(n)$. Con 256 elementos, 8 pasos.

El montículo nunca crece más allá de 256 posiciones: se insertan como máximo 256 hojas y cada unión saca dos nodos y mete uno, así que el tamaño solo baja.

Un detalle que encontró la auditoría. `mont_extraer` hacía `m->n--` antes de comprobar que el montículo tuviera elementos. Con el montículo vacío, el contador habría pasado de 0 a un número enorme (es `size_t`, sin signo) y se habría indexado fuera del arreglo. Hoy no puede pasar, porque el bucle de uniones solo extrae con `m.n > 1`, pero gcc lo advirtió al compilar con `-O2`. Se reescribió para que no pueda ocurrir nunca, sin cambiar el orden de los elementos ni, por tanto, el formato.

2. menor_que — la función más importante del archivo

```
static int menor_que(const NodoHuf *a, const NodoHuf *b) {
    if (a->freq != b->freq) return a->freq < b->freq;
    return a->orden < b->orden;
}
```

Son dos líneas y de ellas depende que el formato funcione.

Si solo se comparara por frecuencia, cuando dos nodos empatan la respuesta sería "ninguno es menor", y el montículo los dejaría en el orden que resultara de los intercambios previos. Ese orden puede cambiar entre ejecuciones, y un orden distinto produce **un árbol distinto con códigos distintos**.

El problema es que el descompresor reconstruye el árbol desde la tabla de frecuencias guardada en la cabecera. Si construye uno diferente, el archivo se descomprime como basura. No da error, no se cae: simplemente escupe bytes incorrectos.

El campo `orden` rompe todos los empates: hojas de 0 a 255 (su propio símbolo), internos de 256 en adelante (un contador). Como no hay dos iguales, la comparación es un **orden total estricto** y el árbol queda unívocamente determinado por las frecuencias.

3. El conteo y la reducción

```
void huf_contar(const unsigned char *datos, size_t n, uint64_t freq[256]) {
    for (size_t i = 0; i < n; i++) freq[datos[i]]++;
}
```

Un bucle trivial, pero importa **lo que no tiene**: ninguna variable global, ningún `static`. Todo el estado entra por parámetro. Por eso tres hilos pueden ejecutarla simultáneamente sobre bloques distintos con arreglos `freq` propios, sin un solo mutex.

```
void huf_acumular(uint64_t dst[256], const uint64_t src[256]) {
    for (int i = 0; i < 256; i++) dst[i] += src[i];
}
```

Esta es la **reducción**: cada hilo funde su conteo local en el global una sola vez. 256 sumas bajo mutex, frente a los millones de bloqueos que harían falta si compartieran el arreglo byte por byte.

Estas dos funciones son la razón por la que paralelizar el conteo no exigió tocar este archivo: ya estaban diseñadas para ello.

4. huf_construir, paso a paso

La arena.

```
h->nodos = malloc(HUF_MAX_NODOS * sizeof(NodoHuf));
```

511 nodos de una vez. El número sale de una propiedad de los árboles binarios: si todos los internos tienen exactamente dos hijos, con k hojas hay $k-1$ internos. Con 256 símbolos máximo, 256 + 255 = 511. Es una cota dura, nunca se excede, así que no hay que redimensionar nada.

Tres ventajas sobre un `malloc` por nodo: una llamada al allocator en vez de 511, los nodos quedan contiguos (mejor localidad al recorrer), y liberar es **un solo** `free` en vez de un recorrido recursivo que puede dejar fugas si se equivoca el orden.

Los nodos se van tomando con `&h->nodos[h->n_nodos++]`, que es un "allocator" de una línea.

Las hojas.

```
for (int s = 0; s < 256; s++) {
    if (freq[s] == 0) continue;
    ...
}
```

El `continue` importa: solo entran los símbolos que **aparecen**. Si se metieran los 256 siempre, los que no aparecen ocuparían lugar en el árbol, empujarían hacia abajo a los demás y alargarían sus códigos. Se comprimiría peor sin ningún motivo.

Las uniones.

```
while (m.n > 1) {
    NodoHuf *a = mont_extraer(&m);   /* izquierdo, bit 0 */
    NodoHuf *b = mont_extraer(&m);   /* derecho,   bit 1 */
    padre->freq = a->freq + b->freq;
    padre->izq = a;  padre->der = b;
    mont_insertar(&m, padre);
}
```

Cada vuelta saca dos y mete uno, así que el montículo baja en uno por iteración y el bucle termina en $k-1$ vueltas.

Que el primero extraído vaya a la izquierda es parte de la convención del formato, igual que el orden de los bits en `bitio`. Invertirlo produciría códigos válidos pero distintos, y de nuevo compresor y descompresor tienen que coincidir.

El caso del símbolo único.

```
if (h->raiz->simbolo >= 0) {
    h->tabla[h->raiz->simbolo].codigo   = 0;
    h->tabla[h->raiz->simbolo].longitud = 1;
    return 0;
}
```

Con una sola hoja, el bucle de uniones nunca se ejecuta y la raíz **es** la hoja. El camino desde la raíz hasta ella mide cero, así que el código natural sería la cadena vacía: imposible de escribir y de leer. Se le fuerza un bit.

Un archivo de un millón de `a` ocupa así un millón de bits (125 KB) en vez de 1 MB. Es el error clásico de las implementaciones de Huffman, y `huf_decodificar` lo maneja de forma simétrica: consume un bit aunque no tenga

ramas que recorrer, o el contador se desincroniza.

5. asignar_codigos — el recorrido

```
        asignar_codigos(h, n->izq, (prefijo << 1) | 0u, prof + 1);
        asignar_codigos(h, n->der, (prefijo << 1) | 1u, prof + 1);
```

El `prefijo << 1` desplaza lo acumulado una posición a la izquierda, abriendo hueco al final, y el `| 0` o `| 1` escribe el bit nuevo ahí. Es exactamente "añadir un dígito por la derecha", como multiplicar por 10 y sumar en decimal.

Al llegar a una hoja, `prefijo` contiene el código completo y `prof` su longitud.

El chequeo de `prof > 64` falla explícitamente en vez de truncar en silencio. Existe un límite teórico: con frecuencias tipo Fibonacci el árbol puede degenerar y un código superar los 64 bits. En la práctica haría falta un archivo de más de un exabyte; con archivos de hasta unos gigabytes la profundidad máxima ronda los 46 bits.

6. Codificar y decodificar

Emitir es una búsqueda en la tabla, O(1):

```
        return bw_bits(w, cod->codigo, cod->longitud);
```

Decodificar es el recorrido inverso:

```
        while (n->simbolo < 0) {
            int b = br_bit(r);
            n = b ? n->der : n->izq;
        }
        return n->simbolo;
```

Se lee un bit, se baja por la rama que indica, y se repite **hasta tocar una hoja**. Aquí es donde se usa la propiedad de prefijo: como todos los símbolos están en hojas, el momento de parar nunca es ambiguo. No hace falta saber de antemano cuántos bits mide el código.

Los chequeos de `b < 0` y `n == NULL` cubren que se acaben los bits a media letra y que el árbol esté corrupto. Sin ellos, un `.huf` dañado provocaría lecturas de memoria inválida.

Este recorrido sigue existiendo, pero desde la auditoría ya no es el camino principal: `huf_formato.c` decodifica casi todos los símbolos con una tabla de búsqueda y solo usa el árbol para los códigos largos (ver Decodificación con tabla de búsqueda).

test_huffman.c — qué verifica cada grupo de pruebas

Prueba	Qué verifica
`abracadabra`	Los códigos son exactamente `a=0`, `c=100`, `d=101`, `b=110`, `r=111`, y 23 bits en total
Propiedad de prefijo	En 200 árboles aleatorios, ningún código es prefijo de otro
Igualdad de Kraft	La suma de 2^(−longitud) vale exactamente 1
Determinismo	Con 66 símbolos empatados, dos construcciones dan tablas idénticas
Ida y vuelta	Codificar y decodificar devuelve el texto original
Símbolo único	Se fuerza el código de 1 bit y la ida y vuelta funciona
Archivo vacío	No es un error; decodificar retorna −1
Datos binarios	Funciona con `0x00` y `0xFF` incluidos
Reducción	Contar por bloques y sumar == contar de una pasada

La de **Kraft** merece atención: un código de prefijo es *completo* —no desperdicia ningún patrón de bits— si y solo si la suma de 2^(−longitud) sobre todas las hojas vale exactamente 1. Huffman siempre produce códigos completos, así que esa identidad tiene que cumplirse. Se calcula con enteros, escalando por 2^L, para que no haya error de

redondeo. Es una verificación matemática de la estructura del árbol, independiente de cualquier ida y vuelta.

La de **determinismo** es la que protege el formato: fuerza el peor escenario posible y comprueba que dos construcciones dan tablas idénticas byte por byte.

Resultado: 23 pruebas, 23 superadas.

Lo que hacen huf_formato.c y huf_formato.h

Qué hace

Define **cómo se ve un archivo `.huf` por dentro** y es el único módulo del compresor que habla con el disco mediante llamadas al sistema. Toma el diccionario que produce `huffman.c` y la mecánica de bits de `bitio.c`, y con eso arma o desarma un archivo completo.

La disposición del archivo (formato HUF2)

Offset	Tamaño	Contenido
0	4	`"HUF2"`, número mágico
4	4	tamaño de bloque en bytes (65 536)
8	8	tamaño original del archivo
16	8	número de bloques
24	2	`n_simbolos`: cuántos valores de byte distintos aparecen
26	9 × `n_simbolos`	pares (byte 1 B, frecuencia 8 B), en orden creciente de byte
`pre`	8 × bloques	índice: bits útiles de cada bloque
...	resto	los bloques comprimidos, uno tras otro

donde `pre = 26 + 9 × n_simbolos`, el tamaño de la cabecera, que calcula `huf_tam_preambulo(freq)`. Como depende de cuántos símbolos aparecen, **el índice ya no empieza en un offset fijo**: el compresor y el lector lo calculan a partir de la tabla.

Por qué la tabla de frecuencias y no el árbol

Serializar el árbol sería más compacto, pero las frecuencias bastan: como `huf_construir` es determinista, el descompresor reconstruye **exactamente el mismo árbol** a partir de las mismas frecuencias. Se guarda el dato más simple y se deja que el algoritmo haga el resto.

Por qué solo los símbolos presentes: de HUF1 a HUF2

La primera versión del formato, **HUF1**, guardaba siempre las 256 frecuencias posibles, de 8 bytes cada una: **2048 bytes fijos** de tabla, aunque el texto usara solo 15 caracteres distintos. Las 241 frecuencias restantes eran ceros que se escribían y se leían para nada.

```
HUF1: [cabecera 24 B][freq[0]=0][freq[1]=0] ... [freq['a']=4120] ... [freq[255]=0]
                     └──────────────────── 256 × 8 = 2048 bytes SIEMPRE ────────────────┘

HUF2: [cabecera 24 B][n=15][('a',4120)][('e',3907)] ... 15 pares
                     └ 2 B ┘└──────────── 15 × 9 = 135 bytes ──────────┘
```

El efecto en la práctica era que el compresor hacía crecer justo los archivos más comunes de un editor de texto: un texto de 712 bytes producía un `.huf` de 2250 bytes, un "ahorro" de **menos 216 %**.

Entrada	HUF1	HUF2	Diferencia
`"hola mundo " × 60` (660 B, 9 símbolos)	más de 2072 B, creció	**378 B**	ahora comprime 42,7 %
Texto repetitivo (108 000 B)	52 839 B	**50 946 B**	1893 bytes menos de cabecera
Un solo símbolo × 50 000	8330 B	**6293 B**	lo mismo: solo cambia la cabecera
Archivo vacío	2072 B	**26 B**	solo la cabecera fija
Binario aleatorio (100 000 B, 256 símbolos)	102 088 B	102 346 B	**258 B peor**, ver abajo

Las alternativas que se descartaron:

- **Un modo "sin comprimir"**: detectar que comprimir no ayuda y guardar el archivo tal cual. Se descartó porque esconde el síntoma sin corregir la causa: el problema no era Huffman, era pagar por 256 símbolos aunque hubiera 15.
- **Códigos canónicos**: guardar solo la longitud del código de cada símbolo, que deja la tabla en 2 bytes por símbolo en vez de 9. Es lo que hacen los compresores reales, pero exige reconstruir los códigos con otro algoritmo y cambiar también `huffman.c`. Se dejó como mejora futura.
- **Frecuencias de 4 bytes**: ahorraría 4 bytes por símbolo, pero mezclaba tamaños de campo y limitaba los archivos a 4 GB. Se mantuvieron 8 bytes, igual que el resto de campos.

Dónde pierde HUF2, y por qué es aceptable. La tabla dispersa cuesta más que la densa cuando aparecen más de 227 símbolos distintos ($26 + 9 \times 228 > 2072$). Con los 256 bytes presentes, la cabecera mide $26 + 9 \times 256 = 2330$ bytes, 258 más que en HUF1; esa es exactamente la diferencia de la última fila. Solo ocurre con datos casi aleatorios, que de todos modos no comprimen.

Lo que HUF2 no resolvió. Los archivos de unas pocas decenas de bytes siguen creciendo, porque cada símbolo distinto cuesta 9 bytes de tabla. El tamaño de un `.huf` de un solo bloque se descompone así:

```
tamaño = 26 (cabecera fija) + 9 × símbolos (tabla) + 8 (índice) + datos codificados
```

Por ejemplo, el texto de 38 bytes de la sesión de `README.md` tiene 15 símbolos distintos y queda en 187 bytes: 26 + 135 de tabla + 8 de índice + 18 de datos. Los datos sí se comprimieron (38 → 18 bytes); lo que pesa es la tabla. Por eso un texto de unas decenas de bytes queda, según cuántos caracteres distintos use, entre unos 190 y unos 225 bytes: textos de 36 y 37 bytes con más símbolos distintos midieron 214 y 223 bytes. HUF2 bajó mucho el tamaño a partir del cual comprimir vale la pena, pero no lo eliminó.

Por qué los pares van en orden creciente de byte. Por dos razones. La cabecera queda determinista, así que el `.huf` sale idéntico con cualquier número de hilos. Y el lector puede detectar símbolos duplicados con una sola comparación contra el anterior.

Costo del cambio de formato. Los `.huf` creados con HUF1 ya no se pueden abrir, y los nuevos no se abren con la versión anterior. El lector reconoce el número mágico `"HUF1"` y lo rechaza con un mensaje claro en lugar de interpretar mal los datos.

Por qué cada bloque está alineado a byte

Un bloque termina casi siempre a mitad de byte. Si el siguiente continuara en ese mismo byte, para codificar el bloque i habría que saber en qué bit terminó el $i-1$: los hilos tendrían que esperarse en cadena y no habría paralelismo.

Rellenando cada bloque hasta completar su último byte, cada uno es **autónomo**. El índice guarda los bits útiles para que el lector ignore el relleno.

Se desperdician a lo sumo 7 bits por bloque. Con bloques de 64 KB, un 0.000013%. A cambio, también se podría descomprimir en paralelo.

Orden de bytes explícito

```
        static void put_u64(unsigned char *p, uint64_t v) {
            for (int i = 0; i < 8; i++) p[i] = (unsigned char)((v >> (8 * i)) & 0xFF);
        }
```

Los enteros se serializan a mano en little-endian, campo por campo, en vez de volcar una `struct` con `memcpy`. Volcar structs es frágil por dos motivos: el compilador puede insertar relleno entre campos, y el archivo dejaría de ser legible en una máquina de endianness distinta. Así el formato queda definido por el estándar, no por el compilador.

Las funciones

El núcleo, solo memoria

```
int huf_comprimir(const unsigned char *datos, size_t n,
                  unsigned char **salida, size_t *salida_n);
int huf_comprimir_con_freq(const unsigned char *datos, size_t n,
                           const uint64_t freq[256],
                           unsigned char **salida, size_t *salida_n);
int huf_descomprimir(const unsigned char *datos, size_t n,
                     unsigned char **salida, size_t *salida_n);
```

No tocan el disco: reciben y devuelven bloques de memoria. Es justo lo que necesita el editor, que le pasará la copia de su documento.

La separación entre `huf_comprimir` y `huf_comprimir_con_freq` es deliberada. La primera cuenta las frecuencias y llama a la segunda. El compresor paralelo cuenta con varios hilos y después llama **a la misma** segunda función. Que la salida paralela sea idéntica a la secuencial deja así de ser una coincidencia: es literalmente el mismo código el que escribe el archivo.

Los envoltorios de archivo

```
int huf_leer_archivo(const char *ruta, unsigned char **datos, size_t *n);
int huf_escribir_archivo(const char *ruta, const unsigned char *datos, size_t n);
```

Aquí están las llamadas al sistema: `open(2)`, `lseek(2)`, `read(2)`, `write(2)`, `close(2)`. Nada de `fopen` ni `fread`.

En `huf_leer_archivo`, `lseek` se usa de forma poco obvia:

```
off_t tam = lseek(fd, 0, SEEK_END);
```

No se está posicionando para leer: se aprovecha que **el valor de retorno de `lseek` es la nueva posición**. Moverse al final y leer ese número da el tamaño en bytes sin llamar a `fstat`. Después un `lseek(fd, 0, SEEK_SET)` devuelve el cursor al inicio.

E/S robusta

```
static ssize_t escribir_todo(int fd, const unsigned char *buf, size_t n) {
    size_t hechos = 0;
    while (hechos < n) {
        ssize_t w = write(fd, buf + hechos, n - hechos);
        if (w == -1) { if (errno == EINTR) continue; return -1; }
        hechos += (size_t)w;
    }
    return (ssize_t)hechos;
}
```

POSIX permite que `write()` transfiera **menos bytes de los solicitados**: por una señal, por disco lleno o por límites del sistema de archivos. Sin este bucle, escribir 800 de 1000 bytes dejaría el archivo truncado y el programa reportaría éxito.

`EINTR` se trata aparte porque no es un fallo: significa que una señal interrumpió la llamada **antes de transferir nada**. `leer_todo` es el espejo, con `if (r == 0) break;` porque en `read` un 0 significa fin de archivo, no error.

Validación paranoica al descomprimir

Todo dato que viene de un archivo es sospechoso: puede estar corrupto o no ser un `.huf` en absoluto. La función `leer_preambulo` valida cada campo **antes** de usarlo para indexar, en este orden:

```
        if (n < HUF_CABECERA_FIJA)                /* ¿caben los 26 bytes fijos?           */
        if (memcmp(d, HUF_MAGICO, 4) != 0)        /* ¿es un .huf HUF2? (rechaza HUF1)     */
        if (c->n_bloques != esperados)            /* ¿bloques coherentes con el tamaño?   */
        if (k > 256)                              /* ¿n_simbolos posible?                 */
        if (n < c->tam_preambulo)                 /* ¿la tabla de símbolos está completa? */
        /* por cada par (s, f) de la tabla: */
        if (s <= previo)                          /* ¿orden creciente, sin duplicados?    */
        if (f == 0)                               /* ¿frecuencia positiva?                */
        if (f > c->tam_original - suma)           /* ¿la suma no se pasa del tamaño?      */
        /* después de la tabla: */
        if (suma != c->tam_original)              /* ¿las frecuencias suman exactamente?  */
        if (c->n_bloques > (n - pre) / 8)         /* ¿el índice está completo?            */
```

Y ya decodificando, para cada bloque:

```
        if (off_carga + bytes > n)                /* ¿el bloque está dentro del archivo?  */
        if (c < 0)                                /* ¿se acabaron los bits a media letra? */
        if (br_restantes(&r) != 0)                /* ¿sobran bits útiles en el bloque?    */
```

Algunas comprobaciones merecen explicación:

- **La suma de frecuencias debe ser exactamente el tamaño original.** Es una verificación cruzada gratuita: si un solo byte de la tabla está alterado, la suma deja de cuadrar y el archivo se rechaza antes de construir el árbol.
- **`f > tam_original - suma` en lugar de `suma + f > tam_original`.** Las dos parecen iguales, pero la segunda puede desbordar con una frecuencia enorme manipulada: `suma + f` daría la vuelta a un número pequeño y pasaría la comprobación. Restar del lado que no puede desbordar evita el problema.
- **`n_bloques > (n - pre) / 8`** por la misma razón: multiplicar `n_bloques * 8` podría desbordar; dividir no.

Sin estas comprobaciones, un `.huf` manipulado provocaría lecturas fuera del buffer. Nueve de las pruebas de `pruebas_huf.sh` verifican exactamente este comportamiento.

Decodificación con tabla de búsqueda

El problema. Descomprimir recorriendo el árbol significa, por cada bit, una llamada a `br_bit` (con una división y un módulo) y un salto a un nodo que puede estar en cualquier parte de la memoria. Un archivo de 64 MB son cientos de millones de llamadas, y la descompresión era la parte más lenta del sistema.

La idea. En vez de leer un bit y bajar un nivel, se miran **12 bits de una vez** y se consulta una tabla que ya sabe qué símbolo empieza con esos bits y cuánto mide su código.

```
bits pendientes:   0 1 1 0 1 0 0 1 1 1 0 1 ...
                   └──────── 12 bits ────────┘ = índice 0x69D
                                 │
                                 ▼
rapida[0x69D] = { simbolo = 'e', longitud = 3 }
                                 │
                                 ▼
se emite 'e' y se avanzan SOLO 3 bits; los otros 9 se vuelven a mirar
```

Cómo se llena la tabla. Un código de longitud L fija los primeros L bits de la ventana; los $12 - L$ restantes pueden valer cualquier cosa. Por eso cada código ocupa $2^{(12 - L)}$ entradas: un código de 3 bits llena 512 entradas, uno de 12 bits llena una sola. Como los códigos de Huffman cumplen la propiedad de prefijo, dos códigos nunca reclaman la misma entrada.

```
        EntradaRapida e = rapida[mirar_bits(datos + off_carga, bytes, r.pos_bit)];
        if (e.longitud != 0 && e.longitud <= br_restantes(&r)) {
            r.pos_bit += e.longitud;          /* un solo acceso: símbolo completo */
            c = e.simbolo;
        } else {
            c = huf_decodificar(&h, &r);      /* código largo o final del bloque: árbol */
        }
```

Las dos condiciones del `if` son las que mantienen la seguridad.

Condición	Cuándo falla	Qué se hace
`e.longitud != 0`	El código mide más de 12 bits y no está en la tabla	Se usa el árbol
`e.longitud <= br_restantes(&r)`	Cerca del final del bloque, la ventana de 12 bits incluye relleno	Se usa el árbol, que se detiene en `total_bits`

Gracias a ese respaldo, la detección de bloques corruptos es exactamente la misma que antes: todo caso dudoso pasa por el árbol y por sus comprobaciones.

Por qué 12 bits.

Ancho	Tamaño de la tabla	Problema
8 bits	256 entradas	Muchos códigos de texto miden más de 8 bits y caerían al árbol
12 bits	4096 entradas × 2 bytes = 8 KB	Cabe en la caché L1 y cubre casi todos los códigos de texto
16 bits	65 536 entradas × 2 bytes = 128 KB	No cabe en L1: cada consulta sería un fallo de caché

Memoria. La tabla se reserva por cada descompresión y se libera en las tres salidas de la función: éxito, error y cancelación.

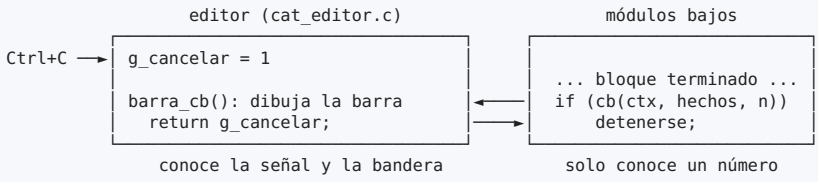
Resultado medido sobre 64 MB: de 1,26 s a 0,53 s solo por la tabla; contando también `-O2`, de 2,34 s a 0,53 s (4,4 veces más rápido). Se verificó con AddressSanitizer y UBSan, y alterando un byte al azar en 300 archivos `.huf` válidos: ninguno produjo un fallo ni un cuelgue.

El callback de progreso

```c
typedef int (*HufProgreso)(void *ctx, uint64_t hechos, uint64_t total);
#define HUF_CANCELADO (-2)
```

Se usa una función en vez de una estructura con mutex para que **ningún módulo bajo tenga que conocer pthread**. `huf_formato.c` y `comprimir.c` solo invocan el callback; quien lo implementa (`cat_editor.c`, que dibuja la barra) decide qué hacer con él. Pasar `NULL` lo desactiva, que es lo que hace el binario `huf`.

El valor de retorno es la vía de cancelación. El callback empezó devolviendo `void`. Para poder cancelar con Ctrl+C pasó a devolver `int`: **0** significa "sigue" y cualquier otro valor, "detente". Cuando el callback pide detenerse, la función deja de trabajar, libera lo suyo y devuelve `HUF_CANCELADO`.



Por qué el compresor no lee la bandera directamente. Habría sido más corto escribir `if (g_cancelar)` dentro de `comprimir.c`, pero entonces el compresor dependería de una variable global del editor y de las señales, y `huf_main.c`, que no tiene Ctrl+C especial, compartiría esa dependencia. Además habría que repetirlo en la descompresión. Con el valor de retorno, `comprimir.c` y `huf_formato.c` siguen sin saber nada del mundo exterior.

Por qué -2 y no -1. Las funciones devuelven `0` si todo salió bien y `-1` si fallaron. Un valor distinto para la cancelación le permite al editor decir "cancelada" en lugar de "falló": que el usuario cancele no es un error del programa.

Desde qué hilo se invoca. Al comprimir, el callback lo llama **siempre el escritor ordenado**, es decir, el hilo que llamó a la función, nunca los trabajadores. Al descomprimir, lo llama el único hilo que hay. Por eso la barra del editor se dibuja sin mutex. Si algún día se moviera la llamada a los trabajadores, la barra necesitaría sincronización y además daría saltos hacia atrás. (El comentario de `huf_formato.h` decía lo contrario y se corrigió.)

Es la misma política que ha guiado todo el proyecto: los módulos de abajo no saben que existen los hilos ni las señales.

Lo que hacen comprimir.c y comprimir.h

Qué hace

Es el único archivo del proyecto que usa hilos. Reparte el trabajo de compresión entre varios hilos y garantiza que el resultado salga en el orden correcto.

`bitio.c` y `huffman.c` no cambiaron para paralelizar. `huf_formato.c` recibió ajustes (la función `huf_comprimir_con_freq`, el callback de progreso y el formato HUF2), pero sigue siendo código secuencial: no incluye `pthread.h` ni sabe que existen los hilos.

La arquitectura: productor / consumidor con ventana acotada

1. Fase de conteo. N hilos cuentan frecuencias sobre tramos disjuntos, cada uno en su propio arreglo local. Al terminar, cada uno funde su conteo en el global una sola vez, bajo mutex.

2. Barrera. El árbol debe construirse con las frecuencias de *todo* el archivo, así que el conteo tiene que terminar por completo antes de empezar a codificar. Aquí la barrera es el `pthread_join` de los hilos de conteo.

3. Fase de codificación. N hilos toman bloques de una cola compartida y los codifican, cada uno en su propio `BitWriter`. Terminan en desorden.

4. Escritor ordenado. El hilo coordinador, que es el mismo que llamó a la función, escribe el bloque *b* en cuanto está listo, en orden estricto. Si el que le toca aún no está, **duerme** en una variable de condición, nunca en un bucle de espera activa. Después de escribir cada bloque, informa el progreso.

```
tiempo ──────────────────────────────────────────────────────────►
trabajador 1   [codifica b0]      [codifica b2]   [codifica b4]
trabajador 2   [codifica b1 ..........]    [codifica b3]
escritor        espera b0 ─► escribe b0 ─► espera b1 ─► escribe b1, b2 ─► ...
                (duerme)      barra 20 %    (duerme)    barra 60 %
```

Los trabajadores terminan en desorden (b2 puede estar antes que b1), pero el archivo sale en orden porque solo el escritor escribe.

La reducción local: la decisión que define la fase 1

La forma ingenua de contar en paralelo es un arreglo global `freq[256]` protegido por un mutex, bloqueándolo cada vez que se ve un carácter. Con un archivo de 10 MB serían **10 millones de bloqueos**, y los hilos pasarían más tiempo esperándose que contando. Paralelizar así sale **más lento** que no paralelizar.

La forma correcta es la reducción local: cada hilo cuenta en su propio arreglo, sin compartir nada y sin ningún mutex. Al terminar, suma su arreglo al global **una sola vez**. Son N bloqueos en total.

```
        static void *hilo_contar(void *p) {
            ArgConteo *a = (ArgConteo *)p;

            /* Fase sin sincronización: el 99.99% del trabajo */
            huf_freq_init(a->local);
            huf_contar(a->datos + a->inicio, a->fin - a->inicio, a->local);

            /* Sección crítica: un único bloqueo por hilo */
            pthread_mutex_lock(a->mtx);
            huf_acumular(a->global, a->local);
            pthread_mutex_unlock(a->mtx);
            return NULL;
        }
```

Dos detalles del `ArgConteo`

Cada hilo necesita **su propia copia** de la estructura. El error clásico es pasarle a todos la dirección de la misma variable del bucle (`&i`): como comparten la dirección, todos acaban leyendo el valor final y procesan el mismo tramo.

El arreglo `local` va **dentro** de la estructura, no en un arreglo global indexado por hilo. Como la estructura mide más

de 2 KB, los arreglos de hilos distintos nunca caen en la misma línea de caché de 64 bytes. Eso evita el **falso compartimiento**: si dos hilos escribieran en la misma línea, el procesador tendría que sincronizarla entre núcleos en cada escritura y el rendimiento se desplomaría sin que exista ningún error lógico.

Por qué una ventana acotada

Si los trabajadores pudieran adelantarse sin límite, todos los bloques comprimidos se acumularían en memoria: con un archivo de 1 GB serían 16384 bloques vivos a la vez.

La ventana impide codificar más allá de `VENTANA` bloques por delante del escritor, lo que deja el uso de memoria en **O(hilos)** en vez de O(tamaño del archivo). Es el clásico búfer acotado, y es la razón de que hagan falta **dos** variables de condición:

Variable	Dirección	Significado
`cond_listo`	trabajador → escritor	"terminé un bloque, revisa si es el tuyo"
`cond_espacio`	escritor → trabajadores	"ya escribí uno, hay hueco en la ventana"

El escritor ordenado

```
        for (uint64_t b = 0; b < n_bloques; b++) {
            pthread_mutex_lock(&t.mtx);
            while (!t.listo[b] && !t.error)
                pthread_cond_wait(&t.cond_listo, &t.mtx);   /* DUERME: 0% de CPU */
            int hubo_error = t.error;
            pthread_mutex_unlock(&t.mtx);
            if (hubo_error) break;

            huf_escribir_fd(fd, t.bw[b].datos, t.bw[b].nbytes);   /* FUERA del mutex */
            bits[b] = t.bw[b].total_bits;
            bw_free(&t.bw[b]);                  /* liberar ya: la ventana depende de esto */

            pthread_mutex_lock(&t.mtx);
            t.proximo_a_escribir = b + 1;
            pthread_cond_broadcast(&t.cond_espacio);          /* hay hueco en la ventana */
            pthread_mutex_unlock(&t.mtx);

            if (cb && cb(ctx, hechos, n) != 0) {              /* progreso, siempre en orden */
                /* El callback pidió cancelar: mismo camino que un write fallido.
                   Se marca t.error = 1 bajo el mutex, se hace broadcast en cond_listo y
                   cond_espacio, se sale del bucle y al final se devuelve HUF_CANCELADO. */
                break;
            }
        }
```

Por qué el `write` va fuera del mutex. `write()` puede tardar milisegundos si el disco está ocupado. Si el escritor lo hiciera con el mutex tomado, ningún trabajador podría marcar su bloque como listo durante ese tiempo y el paralelismo se detendría. Es seguro leer `bw[b]` sin el mutex porque `listo[b]` se marcó bajo el mutex, y eso garantiza que los datos del bloque ya son visibles para el escritor.

Los trabajadores **nunca se duermen esperándose entre sí**: si lo hicieran, no habría paralelismo. El único que duerme es el escritor, y solo porque necesita los bloques en orden.

Cuando `pthread_cond_wait` se ejecuta, el kernel saca al hilo de la cola de ejecución: deja de ser candidato a recibir tiempo de procesador. La alternativa prohibida sería:

```
        while (!listo[b]) { }   /* ESPERA ACTIVA */
```

Ese bucle mantiene al hilo ejecutando millones de iteraciones por segundo. En una máquina de 4 núcleos con 3 trabajadores más el escritor, el escritor le estaría robando un núcleo completo a los que hacen el trabajo real.

Por qué `while` y no `if`

Dos razones. Con `broadcast`, el escritor puede despertarse cuando terminó un bloque que no es el suyo; si fuera un `if`, saldría del bloqueo y escribiría un bloque que no está listo. Y además POSIX permite **despertares espurios**: `pthread_cond_wait` puede retornar sin que nadie haya señalado nada. La regla es absoluta: siempre dentro de un `while` que reevalúa la condición.

Por qué `broadcast` y no `signal`

Puede haber varios hilos dormidos en condvars distintas sobre el mismo mutex. Despertar a todos y que cada uno reevalúe su propia condición es lo seguro; `signal` despierta a uno arbitrario y podría ser el que no debía.

El índice se rellena al final

El índice con los bits de cada bloque va en la cabecera, pero no se conoce hasta haber codificado todo. Se reserva su espacio con ceros, se escriben los bloques, y al terminar se vuelve con `lseek(2)` a sobrescribirlo:

```
        size_t pre = huf_tam_preambulo(freq);   /* depende de cuántos símbolos aparecen */
        ...
        lseek(fd, (off_t)pre, SEEK_SET);
        huf_escribir_fd(fd, idx, tam_indice);
```

Con HUF1 el índice empezaba siempre en el byte 2072. Con HUF2 empieza en `pre`, que solo se conoce después de contar las frecuencias. Por eso la cabecera se escribe **después** de la fase de conteo y no al principio.

```
1. escribir cabecera + índice en ceros:  [HUF2 ... tabla][0 0 0 0 0 0]
2. escribir bloques en orden:            [HUF2 ... tabla][0 0 0 0 0 0][b0][b1][b2]
3. lseek(fd, pre) y reescribir índice:   [HUF2 ... tabla][812 790 403][b0][b1][b2]
                                                          ▲
                                                          offset = pre
```

El número de hilos

```
        long n = sysconf(_SC_NPROCESSORS_ONLN);   /* 1. los núcleos disponibles */
        if (getenv("HILOS")) n = ...;             /* 2. la variable de entorno manda */
        if (n > 256) n = 256;                     /* 3. tope duro */
        if ((size_t)n_hilos > n_bytes) ...;       /* 4. nunca más hilos que bytes */
        if (n_hilos == 1) /* atajo secuencial */;
```

`sysconf(_SC_NPROCESSORS_ONLN)` **no** dice cuántos hilos se pueden crear (serían miles), sino cuántos núcleos hay. Para trabajo limitado por CPU, lanzar más hilos que núcleos solo añade cambios de contexto y desalojos de caché.

La variable de entorno `HILOS` no es un capricho: sin ella no se podría ejecutar la prueba que verifica la ausencia de condiciones de carrera.

El invariante que define la corrección

El `.huf` debe salir **idéntico byte por byte** sin importar cuántos hilos se usen:

```
        HILOS=1 ./huf c entrada.txt a.huf
        HILOS=8 ./huf c entrada.txt b.huf
        cmp a.huf b.huf      # debe pasar en silencio
```

Se cumple porque la suma de frecuencias es asociativa, el árbol es determinista y el escritor impone el orden.

Es un detector de carreras que **no depende de la suerte**, a diferencia de ejecutar el programa y esperar a que falle. Las condiciones de carrera se manifiestan una vez de cada cien ejecuciones; esta prueba, si el resultado depende del número de hilos, falla siempre.

Manejo de errores con hilos

```
        for (int i = 0; i < creados; i++) pthread_join(hilos[i], NULL);
```

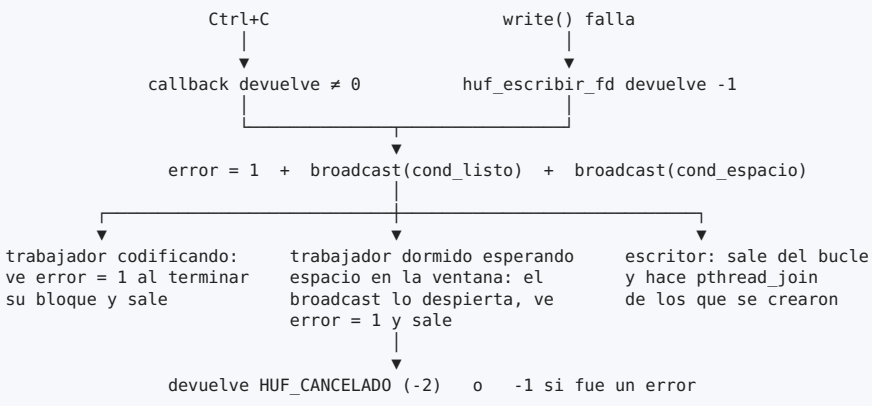
El bucle de `join` usa el contador `creados`, no `n_hilos`. Si la creación del hilo 3 falla, los hilos 1 y 2 ya están vivos y escribiendo en memoria que esta función está a punto de liberar: unirse a ellos es obligatorio incluso en la ruta de error.

Cada trabajador además hace `broadcast` al salir. Si el último en terminar no despertara al escritor, este se quedaría dormido para siempre esperando un bloque que ya nadie va a producir. Ese es el interbloqueo más fácil de introducir

aquí.

La cancelación reutiliza este camino

Cuando el usuario presiona Ctrl+C, el escritor no inventa una forma nueva de detener a los hilos: hace exactamente lo mismo que ante un `write` fallido.



Por qué reutilizarlo. Detener hilos sin interbloqueos es lo más delicado de este archivo, y ese camino ya existía y ya estaba probado. Un segundo mecanismo paralelo habría duplicado esa delicadeza y las oportunidades de equivocarse, por ejemplo olvidar uno de los dos `broadcast` y dejar a un trabajador dormido para siempre en `cond_espacio`.

Por qué no `pthread_cancel`. Cancelar un hilo desde fuera puede cortarlo con el mutex tomado, y entonces nadie más podría tomarlo, o con un `BitWriter` a medio liberar. Con la bandera, cada hilo termina el bloque que tenía en curso y sale por su propio pie, en un punto donde no tiene nada a medias.

La integración con el editor: comandos bloqueantes

Qué hace

Los comandos `comp` y `desc` viven en `cat_editor.c`, en dos funciones: `comprimir_con_barra` y `descomprimir_con_barra`. El hilo principal del editor llama al compresor **directamente** y se queda dentro hasta que termina. Mientras tanto dibuja una barra de progreso; al terminar, `comp` muestra el tamaño original, el comprimido y el porcentaje de ahorro.

```
ubuntu:grande.txt> comp grande.huf
Comprimiendo 4969654 bytes hacia 'grande.huf'. El editor espera.
[comprimiendo] [##############################] 100%
Compresión terminada: 'grande.huf'
  Sin comprimir: 4969654 bytes
  Comprimido:    2555758 bytes
  Ahorro:        48.6%
```

Es una salida real, capturada en una terminal. `grande.txt` es un archivo de prueba de unos 5 MB formado por 700 000 palabras tomadas al azar de una lista de diez, así que es mucho más repetitivo que un texto normal.

Por qué el editor se congela: la decisión y su historia

La primera versión era en segundo plano. `comp` lanzaba la compresión en un hilo aparte, gestionado por un módulo `tarea_comp.c`, y devolvía el prompt de inmediato para que el usuario siguiera editando. Funcionaba, pero cada ventaja de no congelar el editor venía con un problema que había que resolver:

Problema que creaba el hilo de fondo	Cómo se resolvía en `tarea_comp.c`
El usuario podía editar mientras se comprimía	Se entregaba al hilo una **copia** del documento, de uso exclusivo
El `.huf` no incluía lo editado durante la compresión	Se documentaba como una "foto" del momento de lanzar `comp`
Editor e hilo leían el estado y el porcentaje a la vez	Estado compartido con **mutex** y cuatro estados: `INACTIVA → CORRIENDO → OK / ERROR`
El usuario no sabía cómo iba la tarea	Comando extra **`st`** para consultar el porcentaje
Salir con el hilo vivo liberaba memoria que el hilo usaba (*use-after-free*)	Prohibir `q` durante la tarea y llamar a `tarea_esperar()` (`pthread_join`) en tres sitios
El hilo imprimía "45 %" en medio de lo que el usuario tecleaba	Avisos solo cada 10 % por `stderr` y mensaje final aplazado a la siguiente vuelta del bucle
Dos tareas a la vez multiplicaban las rutas de error	Regla de una sola tarea a la vez

Después de discutir con el profesor cuál era el mejor uso de los hilos en el editor, se decidió **congelar el editor mientras comprime**. Las razones:

1. **Desaparece la carrera con la edición.** Mientras `comp` corre nadie puede ejecutar `a`, `i` ni `d`: el documento no cambia, así que no hay nada que proteger.
2. **Se elimina todo un módulo.** Desaparecieron `tarea_comp.c`, el comando `st`, el estado con mutex, la restricción de `q` y la espera final antes de liberar memoria. El editor no tiene carreras de datos propias.
3. **El `.huf` siempre refleja el documento actual**, no una foto del pasado.
4. **La barra no necesita mutex**, porque solo la dibuja el escritor ordenado, que además avanza en orden y nunca retrocede.
5. **La salida es limpia**, porque el usuario no está escribiendo mientras se dibuja la barra.
6. **La concurrencia no se perdió.** Lo que se congela es el bucle de comandos, no la compresión: el conteo y la codificación siguen repartidos entre todos los núcleos. Antes había N trabajadores, el hilo de tarea (que hacía de escritor) y el hilo del editor esperando comandos; ahora el propio hilo del editor hace de escritor y hay un hilo menos.

```
ANTES (segundo plano)
  hilo editor   ──comp──► prompt ── a ── p ── st ── ...      (sigue atendiendo)
                   └──► hilo tarea ──► copia ──► comprimir ──► estado + mutex
                                                  └──► N trabajadores

AHORA (bloqueante)
  hilo editor   ──comp──► comprimir: cuenta, espera bloques, escribe, dibuja barra ──► prompt
                             └──► N trabajadores (conteo y codificación en paralelo)
```

El costo, dicho honestamente. El enunciado hablaba de comprimir sin congelar el editor, así que esta decisión se aparta de su lectura literal. Con documentos que caben en RAM la espera es corta, y la barra mantiene informado al usuario, pero es un punto a defender en la sustentación. La versión en segundo plano se conserva en la rama `backup/antes-cabecera-dispersa`.

Cuánto dura la espera. Con la compilación optimizada (`-O2`) y los cambios de la auditoría, comprimir un archivo de texto de **64 MB** con 4 hilos toma unos **0,54 s**, y descomprimirlo unos **0,55 s**. Un documento de editor, de kilobytes o pocos megabytes, queda muy por debajo. Los tiempos dependen del equipo, así que conviene repetir la medición en el computador de la sustentación.

Y si aun así es demasiado, desde que existe la cancelación el usuario puede abortar con Ctrl+C (ver La cancelación con Ctrl+C). Eso convierte la espera en una elección, no en una trampa.

La justificación completa está en la decisión 10 de `README.md`, en la sección «Por qué el editor se congela mientras comprime».

La cancelación con Ctrl+C

El problema. Congelar el editor tenía un efecto secundario: mientras `comp` corría, la única forma de detenerlo era Ctrl+C, y Ctrl+C **mataba el proceso entero**. El usuario perdía los cambios sin guardar del documento, el `.huf` quedaba a medio escribir con el índice en ceros y los hilos quedaban cortados a mitad de un bloque.

Qué pasa ahora:

Comando	Mensaje	Qué queda en disco	Editor
`comp salida.huf`	`Compresión cancelada: no se modificó ningún archivo.`	Nada nuevo; una `salida.huf` previa queda intacta	Vivo, con el documento y sus cambios sin guardar
`desc x.huf salida`	`Descompresión cancelada: no se escribió '…'`	Nada	Vivo

Las piezas, de arriba abajo

1. El manejador de la señal solo levanta una bandera.

```c
        static volatile sig_atomic_t g_cancelar = 0;

        static void manejador_sigint(int sig) {
            (void)sig;
            g_cancelar = 1;            /* lo ÚNICO que hace */
        }
```

Una señal puede interrumpir el programa en cualquier instrucción: a mitad de un `malloc`, con un mutex tomado, en medio de un `printf`. Si el manejador llamara a `printf` o a `free`, podría reentrar en una función que estaba a medio ejecutarse y corromper su estado. Por eso casi ninguna función es segura dentro de un manejador. Escribir una variable `volatile sig_atomic_t` sí lo es:

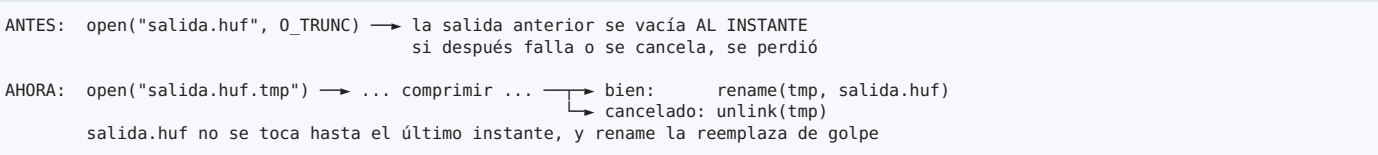
- **`sig_atomic_t`** garantiza que la escritura ocurre de una sola vez: nunca se ve un valor a medias.
- **`volatile`** obliga al compilador a leer la variable de memoria cada vez, en vez de guardarla en un registro y no enterarse nunca del cambio.

2. `barra_cb` traduce la bandera en "detente". Después de dibujar la barra, devuelve 1 si `g_cancelar` está levantada y 0 si no. Los módulos de abajo solo ven ese número (ver El callback de progreso).

3. El compresor se detiene por su camino de error (ver La cancelación reutiliza este camino) y devuelve `HUF_CANCELADO`.

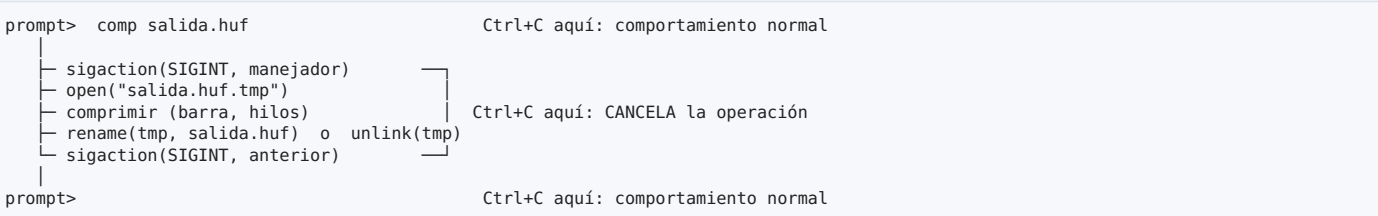
4. `comprimir_con_barra` limpia: borra el temporal con `unlink`, libera la copia del documento, restaura el manejador anterior e informa la cancelación.

La salida se escribe en un temporal



Antes, `open` con `O_TRUNC` vaciaba `salida.huf` en cuanto empezaba la compresión. Si ya existía una versión anterior, se destruía aunque la compresión fallara después. Borrar el `.huf` parcial con `unlink` no arreglaba eso: borraba también lo que el usuario tenía. Con el temporal y `rename`, que es atómico, la salida anterior solo se reemplaza cuando la nueva está completa. Es la misma estrategia que usa `w!` en el editor.

El manejador vive solo durante `comp` y `desc`



Decisión	Por qué
Instalar el manejador solo durante la operación	Con el prompt esperando un comando, Ctrl+C conserva su significado habitual
Instalarlo **antes** de crear el temporal	Si fuera después, habría un instante en que el temporal existe pero Ctrl+C todavía mata el proceso y lo deja sin borrar
`sigaction` y no `signal`	El comportamiento de `sigaction` lo fija el estándar; el de `signal` varía entre sistemas
Sin la opción `SA_RESTART`	Un `read` o `write` bloqueado vuelve con `EINTR` al llegar la señal; `escribir_todo` y `leer_todo` ya lo reintentan

Alternativas descartadas

Alternativa	Por qué no
Limpiar dentro del manejador	`unlink`, `free` y `printf` no son seguros en un manejador interrumpido en cualquier punto
`pthread_cancel` sobre los hilos	Puede cortar un hilo con un mutex tomado o con memoria sin liberar
Que `comprimir.c` lea `g_cancelar`	Acoplaría el compresor a las señales y no resolvería `desc`
`unlink` del `.huf` sin temporal	Destruiría una salida anterior ya existente
Pedir confirmación antes de cancelar	Un manejador no puede leer la entrada de forma segura
Manejador instalado siempre	Cambiaría el significado de Ctrl+C en todo el editor

Cómo se verificó

Herramienta	Escenario	Resultado
`pruebas.sh` (3 pruebas)	`SIGINT` real sobre una compresión de ~80 MB con 1 hilo	Sale el aviso, el editor responde a `q`, no queda ni `.huf` ni temporal
ThreadSanitizer	Cancelar `comp` de 40 MB con 4 hilos y una salida previa existente	0 advertencias; la salida previa quedó idéntica byte a byte
ThreadSanitizer	Cancelar `desc`	0 advertencias
valgrind	Cancelar `comp` de 24 MB con 2 hilos	0 bytes en uso, 0 errores, salida previa intacta, sin temporal

Las pruebas automáticas usan un archivo grande y un solo hilo para que la compresión dure lo suficiente como para alcanzar a cancelarla. El editor se alimenta por una tubería con nombre (`mkfifo`) para poder seguir enviándole comandos después de la señal y comprobar que sigue vivo. Un primer intento con valgrind no valió porque la compresión terminó antes de que llegara la señal; se repitió con un archivo más grande.

Limitaciones de la cancelación

- **Se atiende al terminar el bloque en curso,** de 64 KB, así que es casi inmediata.
- **Durante el conteo de frecuencias,** la bandera se atiende al terminar el primer bloque de la codificación, porque el callback todavía no se ha llamado.
- **`comp` sobrescribe un `salida.huf.tmp` que el usuario ya tuviera.** `w!` resolvió el mismo problema con `mkstemp`; `comp` todavía usa un nombre fijo.
- **La cancelación de `desc` no se ha verificado bajo valgrind,** solo con ThreadSanitizer.

La copia que sigue existiendo

```
        unsigned char *copia = NULL;
        size_t copia_n = 0;
        buf_serializar(&buf, &copia, &copia_n);
        comprimir_con_barra(eargv[1], copia, copia_n, &comprimido_n);   /* la libera adentro */
```

Aunque el editor está detenido, `comp` todavía llama a `buf_serializar`, que arma una copia del documento en memoria (nunca en disco). La razón cambió:

	Antes	Ahora
Para qué existía la copia	Aislar el documento de la edición concurrente	Entregar los bytes **contiguos** que el compresor necesita
Quién la liberaba	El hilo de fondo, al terminar	`comprimir_con_barra`, siempre: salga bien, mal o se cancele

El documento son descriptores de línea dispersos por el heap, sin los `\n`. El compresor recibe un solo bloque lineal de bytes porque lo parte en trozos de 64 KB por posición. `buf_serializar` une las líneas con sus saltos en ese bloque, que es exactamente lo que `w` escribiría en disco. Así, comprimir y luego descomprimir da el mismo archivo que guardar.

La barra de progreso

```
        static int barra_cb(void *ctx, uint64_t hechos, uint64_t total) {
            int pct = (total > 0) ? (int)((hechos * 100) / total) : 100;
            barra_dibujar((Barra *)ctx, pct > 100 ? 100 : pct);
            return g_cancelar ? 1 : 0;     /* 1 = el usuario presionó Ctrl+C */
        }
```

`barra_cb` es el callback que recibe el compresor. Convierte bytes procesados en porcentaje, se lo pasa a `barra_dibujar` y, desde la cancelación, le responde al compresor si debe seguir. `barra_dibujar` tiene dos modos:

Salida	Cómo se ve	Por qué
Terminal (`isatty(STDOUT_FILENO)` es verdadero)	Una barra de 30 celdas que se redibuja en la misma línea con `\r`	El usuario ve el avance sin que la pantalla se llene
Archivo o tubería (`./eafitOS > log.txt`)	Una línea nueva cada vez que el porcentaje cruza un múltiplo de 10	Un `\r` dentro de un archivo deja basura ilegible

Por qué "cruza" y no "es igual a". El progreso avanza de bloque en bloque (64 KB), así que el porcentaje puede saltar, por ejemplo, de 19 a 21 sin pasar por 20. La primera versión solo imprimía cuando el porcentaje era exactamente múltiplo de 10, y en un archivo de 5 MB faltaban las líneas de 20 % y 70 %. Ahora se imprime al cruzar cada múltiplo, aunque el número exacto no aparezca.

`barra_dibujar` ignora las llamadas que repiten el último porcentaje dibujado, así que aunque el compresor informe cientos de bloques, la terminal solo se redibuja cuando el número cambia.

Se descartó una barra con colores porque los códigos de color dependen de la terminal y ensucian la salida redirigida.

Las protecciones

```
        static int mismo_archivo(const char *a, const char *b) {
            if (strcmp(a, b) == 0) return 1;
            struct stat sa, sb;
            return stat(a, &sa) == 0 && stat(b, &sb) == 0 &&
                   sa.st_dev == sb.st_dev && sa.st_ino == sb.st_ino;
        }

        if (mismo_archivo(eargv[1], buf.filename)) {   /* comp hacia el archivo abierto */
            printf("'%s' es el archivo abierto: lo sobrescribirías.\n", eargv[1]);
            ...
        }
```

Situación	Qué hace	Qué evita
`comp` hacia el archivo abierto, con cualquier ruta que lo nombre	Rechaza y sugiere `<archivo>.huf`	Sobrescribir el documento original con datos comprimidos
`desc` hacia el archivo abierto	Rechaza y pide otra salida	Que el disco cambie por debajo del documento en memoria y un `w` posterior lo pise
La compresión falla o se cancela	`unlink(2)` del temporal; la salida anterior no se toca	Que quede un archivo a medio escribir, o que se pierda una salida previa
La descompresión falla o se cancela	No se escribe la salida	`huf_escribir_archivo` solo se llama si la decodificación terminó completa

El primer caso no es teórico: `comp` recibe el nombre de **salida**, y es natural escribir `comp notas.txt` pensando que el

argumento es el archivo a comprimir.

Por qué se compara el inodo y no el texto. La primera versión comparaba los nombres con `strcmp`. La auditoría encontró que con `notas.txt` abierto, `comp ./notas.txt` no coincidía como texto, la protección no actuaba y el `.huf` reemplazaba al documento: el contenido de `notas.txt` pasó a ser `HUF2...`. Lo mismo ocurría con enlaces simbólicos, enlaces duros y rutas con `..`. Todas esas rutas llevan al mismo inodo, y eso es lo que compara `mismo_archivo`: el mismo dispositivo (`st_dev`) y el mismo número de inodo (`st_ino`). Si la salida todavía no existe, `stat` falla y no puede coincidir.

Las syscalls de `comp` y `desc`

Comando	Syscalls, en orden	Dónde
`comp`	`stat` (¿es el archivo abierto?), `sigaction` (instalar manejador), `open` de `salida.huf.tmp`, `write` (cabecera + índice en ceros), `write` por cada bloque, `lseek` + `write` (índice real), `lseek(SEEK_END)` (tamaño final), `close`, `rename` a `salida.huf` si todo salió bien o `unlink` del temporal si falló o se canceló, `sigaction` (restaurar)	`cat_editor.c` y `comprimir.c`
`desc`	`stat`, `sigaction`, `open(O_RDONLY)`, `lseek(SEEK_END)`, `lseek(SEEK_SET)`, `read`, `close`; después, solo si la decodificación terminó, `open(O_WRONLY|O_CREAT|O_TRUNC)`, `write`, `close`; `sigaction` (restaurar)	`cat_editor.c` y `huf_formato.c`

El tamaño final que muestra `comp` se obtiene con `lseek(fd, 0, SEEK_END)` antes de cerrar el descriptor, aprovechando que `lseek` devuelve la nueva posición. Es el mismo truco que usa `huf_leer_archivo` para conocer el tamaño de un archivo sin llamar a `fstat`.

Lo que hace huf_main.c

Es un **binario independiente** para probar el compresor sin pasar por el editor.

```
./huf c <entrada> <salida.huf>    # comprimir
./huf d <entrada.huf> <salida>    # descomprimir
./huf i <archivo.huf>             # información
./huf -v c ...                    # con traza de llamadas al sistema
```

Existe por una razón de método: el ciclo comprimir → descomprimir → `md5sum` dura dos segundos y se puede automatizar. Probarlo desde el bucle del editor obligaría a teclear comandos a mano en cada intento, y durante la caza de un error de bits eso se repite cincuenta veces.

Desde que `make` lo compila junto con `eafitOS`, también es lo que usan los scripts `pruebas_huf.sh` y `pruebas_hilos.sh`. Antes había que compilarlo a mano y, si se olvidaba, esas pruebas fallaban sin que el compresor tuviera ningún error.

El modo `i` lee solo la cabecera e imprime el tamaño original, el comprimido, el porcentaje de ahorro, el número de bloques y cuántos símbolos distintos aparecen. Con HUF2 ese último dato es justamente el que decide el tamaño de la cabecera:

```
$ ./huf i pequeno.huf
archivo:            pequeno.huf
tamaño original:    660 bytes
tamaño comprimido:  378 bytes
ahorro:             42.73%
bloques:            1 de 65536 bytes
símbolos distintos: 9          ← cabecera = 26 + 9 × 9 = 107 bytes
```

Cuando el archivo crece en vez de encogerse, lo advierte. El mensaje atribuye el crecimiento a que el archivo es pequeño, lo cual es cierto para textos cortos pero no para datos aleatorios, donde la causa es que aparecen los 256 símbolos (ver Limitaciones).

Los scripts de prueba

Script	Qué valida	Resultado
`test_bitio`	Empaquetado y desempaquetado de bits	18/18
`test_huffman`	Árbol, códigos, determinismo, casos borde	23/23
`pruebas_huf.sh`	Compresión real de archivos con `md5sum`, y formato HUF2	23/23
`pruebas_hilos.sh`	El invariante de concurrencia	6/6
`pruebas.sh`	Los comandos del editor, la cancelación con Ctrl+C y las correcciones de la auditoría	32/32 (33/33 con `--valgrind`)

`make test` compila y ejecuta las dos primeras. Las demás se ejecutan con `./script.sh`.

pruebas_huf.sh

La verificación central es siempre la misma: comprimir, descomprimir y comparar contra el original. Si coinciden, la recuperación fue exacta byte por byte.

Grupo	Pruebas	Qué cubre
Contenido típico	3	Texto repetitivo, texto natural, binario aleatorio
Casos borde	9	Archivo vacío, un solo byte, un solo símbolo repetido, los 256 valores de byte, las fronteras de bloque en 65 535 / 65 536 / 65 537 bytes, varios bloques completos, bytes nulos mezclados con texto
Rechazo de entradas inválidas	4	Archivo que no es `.huf`, `.huf` truncado, `.huf` con un byte corrupto, archivo inexistente
Cabecera dispersa (HUF2)	7	Un archivo de 660 B ya no crece (queda en 378 B); el vacío ocupa 26 B; se rechazan `n_simbolos` > 256, símbolos duplicados, frecuencias que no suman el tamaño original, tabla truncada y el formato HUF1

Las cinco pruebas de rechazo de HUF2 construyen archivos `.huf` corruptos a propósito: toman uno válido y le modifican un byte concreto de la tabla con Python. Así se prueba cada validación de `leer_preambulo` por separado, en vez de confiar en que un archivo aleatorio dé con ella.

Resultados representativos con el formato actual:

Entrada	Original	Comprimido	Observación
Texto repetitivo	108 000 B	50 946 B	52,8 % de ahorro
`"hola mundo " × 60`	660 B	378 B	42,7 %: con HUF1 crecía
Un solo símbolo × 50 000	50 000 B	6293 B	1 bit por carácter
Archivo vacío	0 B	26 B	solo la cabecera fija
Binario aleatorio	100 000 B	102 346 B	**creció**

Que el binario aleatorio crezca es correcto y vale la pena explicarlo: en datos uniformemente aleatorios los 256 bytes aparecen con la misma frecuencia, todos los códigos miden 8 bits y no hay nada que ahorrar, y encima hay que pagar la tabla de 256 símbolos (2330 bytes). Es una consecuencia del principio del palomar: ningún compresor sin pérdida puede reducir *todas* las entradas posibles.

pruebas_hilos.sh

Comprueba que el `.huf` sale idéntico con 1, 2, 3, 4, 7, 8 y 16 hilos, sobre seis tipos de archivo distintos, y que además se recupera el original. También imprime la tabla de tiempos de la fase de conteo con 1, 2, 4 y 8 hilos.

Es la única evidencia objetiva de ausencia de condiciones de carrera que un evaluador puede reproducir, porque las carreras no se ven leyendo el código ni ejecutándolo una vez.

La tabla de tiempos, medida en el equipo de desarrollo (18 núcleos, archivo de 20 MB):

Hilos	Conteo de frecuencias	Aceleración
1	0,0226 s	1,0×
2	0,0119 s	1,9×
4	0,0072 s	**3,1×**
8	0,0095 s	2,4×



Hasta 4 hilos, cada hilo que se añade casi divide el tiempo. Con 8 es **más lento** que con 4: el conteo de 20 MB dura centésimas de segundo, y crear y unir cada hilo tiene un costo fijo que no se reparte. Con 8 hilos cada uno cuenta solo 2,5 MB y ese costo fijo pesa más que el trabajo que se ahorra. El conteo, además, procesa unos 900 MB/s por hilo, así que nunca fue el cuello de botella.

Esta tabla mide **solo el conteo**, no la compresión completa. Medir la compresión completa con 1, 2, 4 y 8 hilos sigue pendiente, y sería el dato más fuerte para defender la concurrencia en la sustentación.

ThreadSanitizer

ThreadSanitizer es un detector de carreras incluido en el compilador: instrumenta cada acceso a memoria y avisa si dos hilos tocan el mismo dato sin sincronización. Se compila así:

```
gcc -Wall -Wextra -std=gnu99 -g -D_GNU_SOURCE -fsanitize=thread -pthread \
    main.c cat_*.c buffer.c bitio.c huffman.c huf_formato.c comprimir.c -o eafit_tsan
```

Salida vacía significa cero carreras detectadas. Con el código actual, que incluye la cancelación y la auditoría, se verificó el editor completo en tres escenarios:

Escenario	Resultado
Cancelar `comp` sobre 40 MB con 4 hilos, con una salida previa existente	0 advertencias; la salida previa quedó idéntica byte a byte
Comprimir 40 MB con 8 hilos, sin cancelar	0 advertencias
Cancelar `desc`	0 advertencias

El primero es el más exigente: la señal llega con los trabajadores a mitad de bloque y algunos dormidos en `cond_espacio`, que es justo donde un error en el camino de cancelación dejaría un hilo colgado o una carrera.

Valgrind

valgrind ejecuta el programa sobre un procesador simulado y registra cada `malloc` y cada `free`, así que detecta fugas y lecturas de memoria inválida. Con valgrind 3.26.0:

Escenario	Resultado
`./pruebas.sh --valgrind`	33/33, sin fugas
Sesión con `o`, `a`, `comp` y `desc` sobre 400 KB con 4 hilos	0 bytes en uso al salir, 0 errores, ida y vuelta correcta
`comp` cancelado sobre 24 MB con 2 hilos	0 bytes en uso, 0 errores, salida previa intacta, sin temporal
`buf_cargar` con un `malloc` que falla a propósito	Antes de la auditoría se perdían 70 251 bytes; ahora 0

El tercero confirma que la cancelación libera los bloques, la copia serializada, el árbol y la tabla de decodificación, y une todos los hilos. Un primer intento no valió porque la compresión terminó antes de que llegara la señal; se repitió con un archivo más grande.

AddressSanitizer y UBSan

Después de la auditoría se ejecutaron todas las suites con AddressSanitizer (lecturas y escrituras fuera de rango, memoria usada después de liberarla) y UBSan (comportamiento indefinido, como desbordamientos de enteros con signo): sin errores. Durante la propia auditoría se comprimieron y descomprimieron tamaños en los bordes de bloque (0, 1, 2, 255, 256, 257, 65 535, 65 536, 65 537, 131 073, 200 000 y 1 000 000 bytes) con distintos tipos de datos, y se alteró un byte al azar en 300 archivos `.huf` válidos: el descompresor nunca se colgó ni accedió fuera de rango.

Compilar y usar

Compilación

```
        make            # compila ./eafitOS (shell + editor) y ./huf (compresor independiente)
        make test       # pruebas unitarias de bitio y huffman
        make clean      # borra .o y binarios
```

El `Makefile` compila los cuatro módulos del compresor una sola vez y los enlaza en los dos binarios. Todos los objetos dependen de todas las cabeceras: si cambia `huf_formato.h`, se recompila todo, lo que evita mezclar objetos de dos versiones del formato.

Las banderas son `-Wall -Wextra -std=gnu99 -O2 -g -D_GNU_SOURCE -pthread`:

Bandera	Por qué
`-O2`	Antes se compilaba en `-O0`, sin optimizar. Solo con esta bandera, comprimir 64 MB con 1 hilo pasó de 3,35 s a 1,84 s y descomprimir, de 2,34 s a 1,26 s
`-g`	Se conserva para depurar con `gdb` y para que valgrind muestre líneas de código
`-pthread`	Obligatoria: no solo enlaza la biblioteca, también define las macros que hacen seguras ciertas partes de la biblioteca estándar en programas con hilos

Activar `-O2` hizo aparecer dos avisos que en `-O0` no salían: el valor de retorno de `write` ignorado en el editor y el índice de `mont_extraer`. Se corrigieron en lugar de silenciarlos, y la compilación queda sin avisos con `-Wall -Wextra`.

Desde el editor

```
Wordsito> editor
ubuntu:editor> o documento.txt
'documento.txt' cargado en memoria (1225 líneas).

ubuntu:documento.txt> comp documento.txt
'documento.txt' es el archivo abierto: lo sobrescribirías.
'comp' recibe el nombre de SALIDA. Prueba: comp documento.txt.huf

ubuntu:documento.txt> comp documento.huf
Comprimiendo 48225 bytes hacia 'documento.huf'. El editor espera.
[comprimiendo] [##############################] 100%
Compresión terminada: 'documento.huf'
  Sin comprimir: 48225 bytes
  Comprimido:    30629 bytes
  Ahorro:        36.5%

ubuntu:documento.txt> desc documento.huf copia.txt
Descomprimiendo 'documento.huf' hacia 'copia.txt'. El editor espera.
[descomprimiendo] [##############################] 100%
Descompresión terminada: 'copia.txt'

ubuntu:documento.txt> q
```

Es una salida real, capturada en una terminal; `documento.txt` era una copia de 48 225 bytes de la versión anterior de este README. Se añadieron líneas en blanco entre comandos para leerla mejor. (Las trazas `[syscall]` de `o` se omiten aquí. El nombre de la máquina en el prompt depende del equipo.)

Comando	Qué hace
`comp <salida.huf>`	Comprime el documento en memoria. El editor espera y muestra una barra; al final informa los tamaños y el ahorro
`desc <entrada.huf> <salida>`	Descomprime un `.huf` hacia un archivo. El editor espera y muestra una barra

`comp` comprime **lo que está en memoria**, incluidos los cambios todavía no guardados con `w`. Es una consecuencia directa de que el editor trabaje sobre la RAM.

El editor rechaza comprimir o descomprimir sobre el archivo que está abierto, para no destruir el documento del usuario.

Desde la terminal

```
./huf c notas.txt notas.huf          # comprimir
./huf d notas.huf notas_copia.txt    # descomprimir
./huf i notas.huf                    # ver la cabecera
HILOS=4 ./huf -v c grande.txt g.huf  # 4 hilos, con traza de syscalls y tiempos por fase
cmp notas.txt notas_copia.txt        # en silencio = recuperación exacta
```

Decisiones de diseño transversales

Los módulos bajos no conocen los hilos ni las señales. `bitio.c`, `huffman.c` y `huf_formato.c` no incluyen `pthread.h`. Los dos primeros no cambiaron al paralelizar; `huf_formato.c` solo recibió ajustes que no lo atan a los hilos (`huf_comprimir_con_freq`, el callback y HUF2). Se escribieron sin estado global, recibiendo todo por parámetro, y eso bastó. La cancelación respetó la misma regla: los módulos solo ven que el callback devolvió un número distinto de cero.

Primero correcto, después rápido. El compresor se construyó y validó entero en versión secuencial antes de introducir un solo hilo. Así, cuando algo falló al paralelizar, el algoritmo ya estaba descartado como sospechoso.

Una sola definición de cada cosa. El formato del archivo se declara en un único lugar (`huf_tam_preambulo`, `huf_escribir_preambulo`) y el compresor paralelo lo reutiliza. Duplicarlo habría funcionado el primer día y roto el formato en silencio en cuanto las dos copias divergieran. El paso de HUF1 a HUF2 lo demostró: el formato cambió en `huf_formato.c` y el compresor paralelo solo tuvo que pedir el nuevo tamaño de cabecera.

Corregir la causa, no el síntoma. Cuando los archivos pequeños crecían, se cambió la tabla que los hacía crecer (HUF2) en lugar de añadir un modo que guardara el archivo sin comprimir.

La concurrencia donde da velocidad, no donde da complejidad. Los hilos están en el conteo y la codificación, que es donde se usa la CPU. Se quitaron del bucle de comandos del editor, donde solo aportaban estado compartido, un comando extra y riesgo de *use-after-free*.

Reutilizar el camino que ya funciona. La cancelación no inventó una forma nueva de detener los hilos: entra por el camino de error que ya existía y ya estaba probado. Menos código nuevo en la parte más delicada del programa es menos riesgo.

Las pruebas prueban propiedades, no ejecuciones. El invariante de que la salida no depende del número de hilos, la igualdad de Kraft, la propiedad de prefijo: son afirmaciones verificables, no "lo ejecuté y no se cayó". Del mismo modo, cada corrección de la auditoría tiene una prueba que **falla con el código anterior y pasa con el nuevo**; una prueba que pasa con las dos versiones no demuestra nada.

Optimizar sin cambiar el resultado. Los cambios de rendimiento de la auditoría (`bw_bits` por bytes, tabla de decodificación, `-O2`) se aceptaron solo porque el `.huf` resultante es idéntico byte a byte al de antes. Si una optimización hubiera cambiado el formato, habría sido un cambio de formato, no una optimización.

Verificar antes de afirmar. Las cifras de valgrind, ThreadSanitizer y tiempos de este documento solo aparecen porque se ejecutaron y se tiene su salida.

Limitaciones conocidas

El editor no responde mientras comprime. Es la consecuencia de la decisión de bloquear. La espera es corta y se puede cancelar con Ctrl+C, pero el enunciado pedía comprimir sin congelar el editor, y esta diferencia hay que defenderla en la sustentación.

La cancelación tiene cuatro límites, descritos en Limitaciones de la cancelación: espera a que termine el bloque en curso, durante el conteo se atiende al empezar la codificación, `comp` sobrescribe un `salida.huf.tmp` previo del usuario, y la cancelación de `desc` no se ha verificado bajo valgrind.

Los archivos diminutos todavía crecen. HUF2 resolvió el caso de los archivos de cientos de bytes, pero uno de unas decenas de bytes sigue saliendo más grande: queda entre unos 190 y unos 225 bytes según cuántos caracteres distintos use (38 → 187 bytes con 15 símbolos en la sesión de ejemplo), porque cada símbolo distinto cuesta 9 bytes de tabla. Los códigos canónicos dejarían la tabla en 2 bytes por símbolo.

Los datos aleatorios crecen un poco más que con HUF1. Con los 256 símbolos presentes, la tabla dispersa mide 2330 bytes frente a los 2072 de la densa. Son 258 bytes extra en un caso que de todos modos no comprime.

La descompresión es secuencial. El formato ya lo permitiría: cada bloque es autónomo y el índice da su posición, así que bastaría sumar los tamaños para saber dónde empieza cada uno y repartirlos entre hilos. La auditoría lo propuso (punto 12) y se decidió **no** hacerlo: no corrige ningún fallo, es el cambio más grande de la lista y obligaría a repetir toda la verificación con ThreadSanitizer. Además, con la tabla de búsqueda, descomprimir 64 MB ya toma unos 0,55 s, casi lo mismo que comprimir con 4 hilos.

El documento completo vive en RAM, y `comp` lo duplica. Un archivo más grande que la memoria disponible no se puede abrir ni comprimir desde el editor. Además, durante `comp` conviven el documento y su copia serializada, así que el pico de memoria es aproximadamente el doble del tamaño del documento.

Formatos incompatibles. Los `.huf` creados con HUF1 ya no se pueden abrir con esta versión.